

The problem generics solve

Start with a function that prints a slice of ints:

```
func printSlice(items []int) {  
    for _, item := range items {  
        fmt.Println(item)  
    }  
}
```

Now you need the same thing for `[]string`. The body is identical — only the parameter type changes. So you'd write `printStringSlice([]string)`. Then `[]bool` comes along, and you write a third. The logic is copy-pasted three times; only the signature differs. That's a direct DRY violation, and it's the exact pain generics were added to fix (Go 1.18, so any modern toolchain has them).

Before 1.18 your only escape hatches were `interface{}` (lose all type safety, need type assertions everywhere) or code generation. Generics give you one function that the compiler specializes per type, *keeping* type safety.

Type parameters

```
func printSlice[T any](items []T) {  
    for _, item := range items {  
        fmt.Println(item)  
    }  
}
```

The `[T any]` is the **type parameter list**. `T` is a placeholder for a type (convention is a single capital letter — `T`, `K`, `V`); `any` is its **constraint** — the set of types `T` is allowed to be. `any` means "literally any type."

Then `items []T` uses `T` as if it were a real type. One definition now serves ints, strings, bools, anything.

Type inference

When you call it:

```
nums := []int{1, 2, 3}
printSlice(nums)    // T inferred as int
```

```
names := []string{"go", "typescript"}
printSlice(names)   // T inferred as string
```

You don't write `printSlice[int](nums)`. The compiler looks at the argument's type and infers `T` for you. You *can* write it explicitly, but inference handles the common case. This is the key ergonomic win — at the call site it reads like a normal function.

any is just an alias

The transcript mentions you can write `interface{}` instead of `any`:

```
func printSlice[T interface{}](items []T) { ... }
```

These are identical — `any` is literally a built-in alias for `interface{}` added in 1.18. Use `any`; it's cleaner. The point being made is that an empty interface as a constraint means "no restriction."

Constraining the type set

`any` is often too loose — it lets *anything* through, which is usually not what you want. You can restrict `T` to a **union** of specific types using `|`:

```
func printSlice[T int | string](items []T) {
    for _, item := range items {
        fmt.Println(item)
    }
}
```

Now `[]int` and `[]string` work, but `[]bool` is a **compile-time error** — the compiler rejects it before the program runs. To allow bools you extend the union:

```
func printSlice[T int | string | bool](items []T) { ... }
```

This is the real value over `interface{}`: you get to define exactly which types are legal, and violations are caught at compile time, not at runtime via a panic.

(In real code these unions are usually extracted into a named constraint interface, e.g. `type Ordered interface { ~int | ~float64 | ~string | ... }`, but the transcript keeps them inline.)

Generics on structs

Functions are the common case, but structs work too. Take a stack (LIFO — last in, first out):

```
type Stack struct {
    elements []int
}

myStack := Stack{
    elements: []int{1, 2, 3},
}
```

This stack only holds ints. To reuse it for strings you'd duplicate the type — same problem as before. Make it generic:

```
type Stack[T any] struct {
    elements []T
}
```

The crucial difference from functions: **structs require explicit instantiation**. There's no argument for the compiler to infer `T` from at the type declaration, so this fails:

```
myStack := Stack{ ... }    // error: cannot use generic type Stack[T] without instantiation
```

You must supply the type argument in the brackets:

```
myStack := Stack[string]{
    elements: []string{"golang"},
}
```

`Stack[string]` is now a concrete type whose `elements` field is `[]string`. That's the syntax to remember: functions infer, struct types make you spell it out.

The `comparable` constraint

Instead of listing every type in a union, Go ships a built-in constraint called `comparable`:

```
func printSlice[T comparable](items []T) { ... }
```

`comparable` is the set of all types that can be used with `==` and `!=`. That covers booleans, numbers, strings, pointers, channels, interfaces, and arrays/structs *whose elements/fields are themselves comparable*. So a struct of comparable fields qualifies; a struct containing a slice does not (slices aren't comparable).

Use it when your logic needs equality checks (deduplication, lookups, set membership). Don't use it as a generic "allow everything" — that's what `any` is for. `comparable` specifically promises `==` works.

Multiple type parameters

You're not limited to one. Separate them with commas:

```
func printSlice[T any, V string](items []T, name V) {  
    for _, item := range items {  
        fmt.Println(item, name)  
    }  
}
```

```
printSlice(nums, "john") // T inferred as int, V as string
```

`T` and `V` are independent placeholders with independent constraints. This is how built-ins like `map[K]V` operations get expressed — one type param for keys, one for values.

The mental model: a type parameter is a variable that holds a *type* instead of a value, and the constraint defines the legal set of types that variable can take. Functions infer those types from arguments; struct types need you to provide them explicitly. Pick the tightest constraint that still lets your code work — `comparable` if you need `==`, a union if you need a specific small set, `any` only when you truly mean anything.